

Session 2: Persistent Data Structures

Léon Gondelman

September 9, 2026

SW7 Programming Paradigms Course
Aalborg University @ Copenhagen

Roadmap for today

Lecture.

- *Persistent data structures*: ephemeral versus persistent structures, the checkpoint trick and the case in which it fails, immutability $\Rightarrow$ safe sharing $\Rightarrow$ persistence, in Java and in Python.
- *A first look at OCaml*: a language in which persistence is the default, and in which the distinction is visible *in the types*.

Lab. We build a *persistent queue* from two lists, and place it where both of its properties earn their keep — as the frontier of a search:

- with the queue as frontier, the search is **breadth-first**;
- exchanging it for today's persistent *stack* makes the same search **depth-first**, without a single line of the algorithm changing;
- and since no version is ever destroyed, the history of the search remains available: we may return to any earlier step and ask what the frontier held.

Part 1: Persistent Data Structures

Data structure = data + structural relations.

Computer programs usually operate on tables of information. In most cases these tables are not simply amorphous masses of numerical values; they involve important structural relationships between the data elements. In order to use a computer properly, we need to understand the structural relationships present within data, as well as the basic techniques for representing and manipulating such structure within a computer.

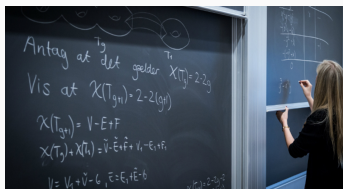
D. E. Knuth, The Art of Computer Programming, vol 1, chap 2,
“Information structures”, 1968.

An algorithm is like a cooking recipe!



- Input data and intermediate results are not preserved during computation.
- Fundamental formalism: the Turing machine.

An algorithm is a mathematical definition



- Computation seen as a sequence of steps constructing the final result, without ever erasing inputs or intermediate results.
- Fundamental formalisms: general recursive functions, λ -calculus, rewriting systems, formal semantics.
- “Declarative” programming (as opposed to “imperative”): functional, logic, constraint-based, etc.

What algorithms for declarative programming?

- Declarative programming is reputed inefficient because it cannot use *ephemeral* (in-place modified) data structures (e.g. arrays).
- **Persistent data structures** address this criticism:
 - Their interface only exposes operations that do not modify the structure in place, but instead return **new** modified structures.
 - Their implementation achieves, or comes close to, the complexity of the best known ephemeral structures.

The relevance of persistence for imperative programming

Even in imperative programming and classical algorithms, persistent data structures can be relevant:

- **Backtracking** (paths exploration, checkpoints) are trivial.
- One can keep **the complete history of updates** of the structure.
- The absence of in-place modification allows **memory sharing** between successive versions, enabling a compact representation of history.
- **concurrency** (e.g. immutable versions can be shared across threads without locks)

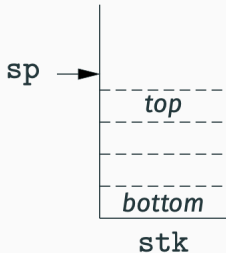
Example: A stack



Operations:

- `init` empties the stack
- `push(v)` pushes `v` on the top
- `top` returns the value at the top
- `pop` removes the value at the top

An ephemeral stack implemented with an array



```
int stk[SIZE];
int sp;

void init(void) { sp = 0; }

void push(int v) {
    assert (sp < SIZE);
    stk[sp] = v; sp = sp + 1;
}

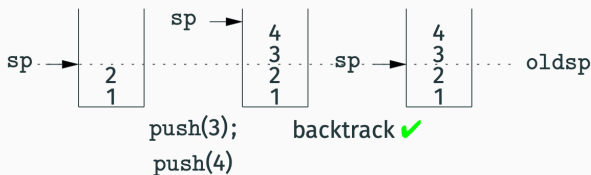
int top(void) {
    assert (sp > 0); return stk[sp - 1];
}

void pop(void) {
    assert (sp > 0); sp = sp - 1;
}
```

Backtracking (checkpointing & backtracking)

Efficient but incomplete approach:

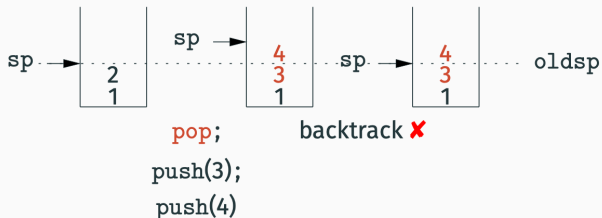
- Checkpoint: `oldsp = sp` (save the stack pointer)
- Backtrack: `sp = oldsp` (restore the stack pointer)



Backtracking (checkpointing & backtracking)

Efficient but incomplete approach:

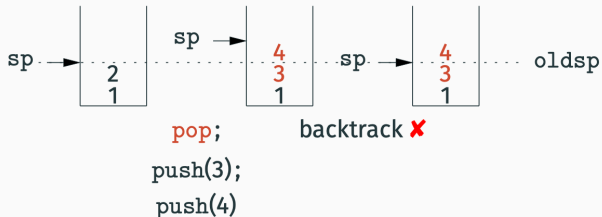
- Checkpoint: `oldsp = sp` (save the stack pointer)
- Backtrack: `sp = oldsp` (restore the stack pointer)



Backtracking (checkpointing & backtracking)

Efficient but incomplete approach:

- Checkpoint: `oldsp = sp` (save the stack pointer)
- Backtrack: `sp = oldsp` (restore the stack pointer)



Always correct but inefficient alternative:
copy `stk[0..sp]` to another array.

A persistent stack implemented as a linked list



```
final class Stack {  
    private final int hd;  
    private final Stack tl;  
    private Stack(int v, Stack s) { hd = v; tl = s; }  
  
    static final Stack empty = null;  
    static Stack push(int v, Stack s) { return new Stack(v, s); }  
    static int top (Stack s) { return s.hd; }  
    static Stack pop (Stack s) { return s.tl; }  
}
```

The interface of the data structure changes: push and pop *return* the new stack value (no in-place mutation).

The implementation relies on the language memory management (GC) to reclaim list cells after pop.

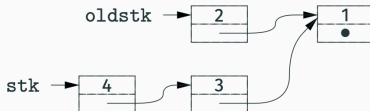
Note every `private` and every `final`. We come back to why.

Persistent stacks and backtracking



```
stk = Stack.push(2, Stack.push(1, Stack.empty));
```

Persistent stacks and backtracking



```
stk = Stack.push(2, Stack.push(1, Stack.empty));
```

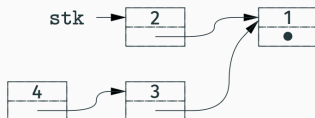
```
// Checkpoint
```

```
oldstk = stk;
```

```
// Work
```

```
stk = Stack.push(4, Stack.push(3, Stack.pop(stk)));
```


Persistent stacks and backtracking



```
stk = Stack.push(2, Stack.push(1, Stack.empty));
```

```
// Checkpoint
```

```
oldstk = stk;
```

```
// Work
```

```
stk = Stack.push(4, Stack.push(3, Stack.pop(stk)));
```

```
// Backtrack
```

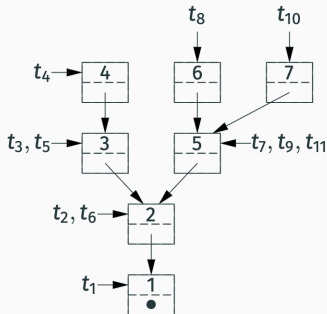
```
stk = oldstk;
```

Persistent stacks and memory sharing

Key observation: a stack produced by push or pop **shares** all but one node with the previous stack.

This enables keeping *all* successive states economically, so that total memory for a sequence of N pushes and M pops is about N nodes.

```
t1 = push(1, empty)
t2 = push(2, t1)
t3 = push(3, t2)
t4 = push(4, t3)
t5 = pop(t4)
t6 = pop(t5)
t7 = push(5, t6)
t8 = push(6, t7)
t9 = pop(t8)
t10 = push(7, t9)
t11 = pop(t10)
```



Why private? Why final?

Two keywords we put on the Java linked-list stack, and never justified.

```
Stack v1 = push(3, push(2, push(1, empty))); // [3, 2, 1]
Stack v2 = pop(v1);                          // [2, 1]
        // v2 IS the second node of v1 -- the two versions SHARE it

v2.tl = null; // if tl were public and non-final
```

What is v1 now?

Immutability, Sharing, Persistence

- **Immutability:** operations never overwrite state; they *return a new value*.
- **Structural sharing:** a new version *reuses the unchanged parts* of the old one — a new node whose tail *is* the old stack.
- **Safe sharing:** no version can be changed *through another version's pointer*. Sharing on its own is just aliasing — the previous slide. *Immutability buys this outright*, by making the change impossible.
- **Persistence:** every old version stays valid and reachable.

immutability $\Rightarrow$ safe sharing $\Rightarrow$ persistence

An implication, not an equivalence — next slide.

Immutability is one route, not the only one

Safe sharing asks for one thing: *no version may be changed through another version's pointer*. There is more than one way to arrange that.

Route	Safe because	Checked by
Immutability	nothing ever changes	compiler: <code>final</code> , <code>frozen</code>
Encapsulation	only the owner may change it	compiler, for clients
Ownership	aliasing and mutation never meet	compiler (Rust)
Benign effects	it changes, nobody can tell	us, by argument
Copy-on-write	a writer gets its own copy	the runtime

We take the first: the compiler checks it, and it keeps *structural* sharing — copy-on-write copies everything on the first write.

We will need the fourth. Session 5 repairs today's queue with a thunk that overwrites itself once forced; session 7 compresses paths in a persistent Union-Find. Both mutate; neither can be caught at it.

If immutability or safe sharing is missing

Sharing without immutability $\Rightarrow$ *corruption*.

```
v2.t1 = null;    // v1 was [3,2,1] -- now [3,2]
```

An old version changed although nobody touched it. Restore that checkpoint while backtracking, and we resume from a state that never existed.

Immutability without sharing $\Rightarrow$ *copying*. To keep a checkpoint we must copy the whole structure: a stack of 10^6 nodes costs 10^6 nodes per checkpoint instead of one pointer. Backtracking degrades from $O(1)$ to $O(n)$ per step.

We need both: drop either one and persistence becomes either *wrong* or *unaffordable*.

Java: why `private` and `final`?

Back to the open question: `v2.tl = null` does not compile. That is the whole point.

- `private` fields: encapsulates representation — clients cannot mutate internals.
- `final` **fields**: enforce immutability after construction $\Rightarrow$ not even Stack itself can rewrite a node once it exists, and the compiler checks it.
- `final` **class** (optional but recommended): prevents subclassing that could reintroduce mutability or break invariants.

Bonus, and a session 7 teaser: `final` also makes these fields safely visible to other threads, with no locking.

Python: tuples-as-cons — immutability & sharing

In Python once tuples are created, their items can't be updated.
Stack as nested tuples: (hd, tl), where tl is another pair or None.

```
EMPTY = None
```

```
def push(v, s):      # O(1)
    return (v, s)    # NEW tuple; 's' is SHARED (structural sharing)
```

```
def top(s):
    if s is None: raise IndexError("empty")
    return s[0]
```

```
def pop(s):          # O(1)
    if s is None: raise IndexError("empty")
    return s[1]
```

```
# Example: versions share structure
```

```
s1 = push(1, EMPTY)      # [1]
s2 = push(2, s1)         # [2,1] (shares node '1')
s3 = push(3, s2)         # [3,2,1] (shares tail [2,1])
```

```
# s2[1] = None          # -> TypeError (tuples are immutable)
```

Sharing: push creates 1 new pair & reuses the whole tail $\Rightarrow O(1)$ upd.

Python: immutable cons-list (structural sharing)

A persistent stack needs nodes that can never change once built. Python says so in one word: frozen.

```
from dataclasses import dataclass
from typing import Optional

@dataclass(frozen=True)
class Stack:
    hd: int
    tl: "Optional[Stack]" = None

def push(v, s): return Stack(v, s)    # one new node; tail s is SHARED
def pop(s):
    if s is None: raise IndexError("empty")
    return s.tl

s1 = push(1, None); s2 = push(2, s1); s3 = push(3, s2)
s2.tl = None          # FrozenInstanceError -- s3 is protected
```

Same story as in Java, one line shorter: `frozen=True` generates the `__setattr__` that refuses the write. Without it, `s2.tl = None` would retroactively change `s3`, which shares `s2` as its tail.

Purely Functional Data Structures

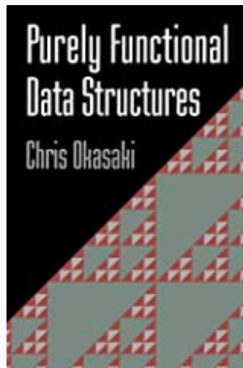
Chris Okasaki
September 1996
CMU-CS-96-177

School of Computer Science
Carnegie Mellon University
Pittsburgh, PA 15213

*Submitted in partial fulfillment of the requirements
for the degree of Doctor of Philosophy.*

Thesis Committee:

Peter Lee, Chair
Robert Harper
Daniel Sleator
Robert Tarjan, Princeton University



Systematic reformulation of advanced structures and algorithms in purely functional style.

In particular, making use of lazy evaluation and amortized algorithm analysis techniques in a novel way.

Some languages make persistence the default

Everything so far was persistence *against* the grain of the language: Java needed `private` and `final`, Python needed `frozen`. Some languages start from the other end.

Lisp / Scheme (1958 onwards) — the cons cell is immutable by convention, and the whole language is built on it:

```
(define s1 (cons 1 '()))      ; [1]
(define s2 (cons 2 s1))      ; [2,1] -- s1 is SHARED, not copied
```

Haskell (1990) — there is no assignment to take away. Every value is immutable, so every structure is persistent whether we intended it or not:

```
s1 = [1]
s2 = 2 : s1    -- the same cons, spelled with a colon
```

The same data structure, and no keyword required: the default has done the work for us.

OCaml: the stack, one more time

Our language for the remainder of the course. Immutable by default but — unlike Haskell — it allows us to *choose*, which is precisely the subject of this course.

The persistent stack, in five lines:

```
type 'a t = 'a list

let empty = []
let push v s = v :: s
let pop s = match s with [] -> None | v :: rest -> Some (v, rest)
let peek s = match s with [] -> None | v :: _ -> Some v
```

Reading it: `let` binds a name; `'a` is Java's `<T>`; `[]` is the empty list and `v :: s` puts `v` in front of `s`; `match` performs case analysis on the shape of a value — the switch statement we always wanted.

`v :: s` *is* the sharing diagram we drew earlier — one new cell, and the old stack reused entire.

Part 2: Introduction to OCaml

A general-purpose language, *statically and strongly typed*, from the ML family (with Standard ML and F#).

- Designed and implemented at Inria by **Xavier Leroy** and others — the same course we followed for the first half of today.
- Types are *checked* everywhere but *written* almost nowhere: the compiler infers them.
- Compiled to native code, and garbage collected.
- Immutable by default — yet mutation is available when we ask for it. That last point is why it belongs in this course.

A language that has an opinion, but still lets us choose.

Who writes OCaml

- **Jane Street** — a trading firm whose entire system is OCaml; one of the largest single codebases in any typed functional language.
- **Proof assistants and verification tools:** Rocq (until recently Coq), Why3, Frama-C, Alt-Ergo — the tools used to prove that programs do what they claim.
- **Static analysis:** Astrée, used on Airbus flight-control software; Infer and Flow at Meta; Semgrep.
- **Systems:** the Tezos blockchain, the Unison file synchroniser, MirageOS unikernels.

The pattern is not an accident: OCaml turns up where being wrong is expensive.

let: definitions, and the types we never write

```
let x = 40 + 2                (* a value *)

let double n = 2 * n          (* a function: no braces, no return *)

let area r =
  let pi = 3.14159 in         (* a LOCAL binding, in force below *)
  pi *. r *. r                (* *. is multiplication on floats *)
```

There is no return: the value of the body *is* the result.

In the interactive toplevel the compiler tells us what it worked out:

```
# let double n = 2 * n ;;
val double : int -> int = <fun>
```

We wrote no type at all, and yet double is fully typed.

OCaml is not “the functional language”

Mutable state is available — we simply have to ask for it by name.

```
let counter = ref 0          (* a mutable cell *)

let tick () =
  counter := !counter + 1;    (* := writes, ! reads *)
  !counter
```

tick takes (), the single value of type unit — OCaml’s way of saying *no interesting argument*, and the return type of anything whose purpose is an effect.

The standard library ships a mutable Stack, Queue and Hashtbl beside the immutable List and Map. *The paradigm is a choice made inside the language, not by it.*

Lists: a linked list with the links hidden

```
let nothing = []           (* the empty list *)
let l  = [1; 2; 3]         (* a literal -- note the SEMICOLONS *)
let l' = 1 :: 2 :: 3 :: [] (* exactly the same list *)
```

A list is a chain of cells, each holding a value and a pointer to the rest — the linked list from the first half of today. The difference is that OCaml never hands us the pointer: there is no `.next` to assign to, and no assignment for list cells anywhere in the language.

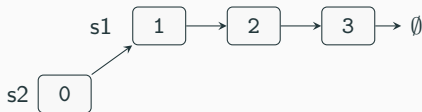
So a list behaves as a Python *tuple* behaves — we may read it and build new ones from it, but we cannot change one. The nested pairs we built by hand in Python are what OCaml simply calls a list.

Careful: `[1, 2, 3]` is not an error. It is a list of *one* element, the triple `(1, 2, 3)` — and the complaint arrives later, somewhere else.

Lists: immutable, and shared

```
let s1 = [1; 2; 3]      (* separator is ; not , *)
let s2 = 0 :: s1        (* [0; 1; 2; 3] *)
```

s2 is one new cell whose tail *is* s1. Nothing was copied, and nothing can be modified: there is no assignment to a list cell in the language at all.



This is the sharing diagram from the first half, now built into the language.

match: asking what shape a value has

```
let rec sum l =  
  match l with  
  | []          -> 0  
  | x :: rest  -> x + sum rest
```

A list is either empty or a head followed by a tail; `match` names those two cases and binds `x` and `rest` in the second.

Two things a `switch` never did for us:

- it *destructures* — `x` and `rest` come out of the value, no accessors needed;
- it is *checked* — omit a case and the compiler says which one. Session 3 makes that guarantee the whole point.

let rec: recursion

A function that calls itself must say so: `let rec`.

```
let rec fact n =  
  if n <= 1 then 1 else n * fact (n - 1)  
  
let rec length l =  
  match l with  
  | []      -> 0  
  | _ :: tl -> 1 + length tl
```

On numbers we recurse towards zero; on lists we recurse towards the empty list. The shape of the data decides the shape of the function — an idea we take much further in session 3. *Loops are not forbidden; they are simply not the first thing we reach for.*

Records: the type our queue will have

Given to us in the lab. A record groups several values under names:

```
type 'a t = { front : 'a list; back : 'a list }  
  
let empty = { front = []; back = [] }      (* construction *)  
let is_empty q = q.front = []             (* field access *)
```

And a shorthand worth recognising before we meet it:

```
{ front; back }      (* means { front = front; back = back } *)
```

'a is the type variable — Java's <T>. A record is immutable unless a field is declared mutable, so every operation on this queue necessarily *returns a new one*.

option: OCaml has no null

What should dequeue return on an empty queue? OCaml refuses to let us answer “nothing, silently”.

```
val dequeue : 'a t -> ('a * 'a t) option
```

A value of type `'a option` is either `None` or `Some v`. The empty case is *in the type*, so a caller cannot overlook it: the value is not reachable until the `None` case has been handled.

Python `raise IndexError`

Java `throw new NoSuchElementException`

OCaml `None` — checked at compile time

The same decision, made three times; only one of them is enforced.

A record may be mutable — and the paradigm changes

A field marked mutable can be assigned with `<-`. Here is the same stack, written imperatively:

```
type 'a stack = { mutable elems : 'a list }

let create () = { elems = [] }
let push v s  = s.elems <- v :: s.elems
let pop s =
  match s.elems with
  | []          -> None
  | v :: rest   -> s.elems <- rest; Some v
```

Same language, same data, same operations. The whole difference is in one type:

imperative	<code>val push : 'a -> 'a stack -> unit</code>
persistent	<code>val push : 'a -> 'a t -> 'a t</code>

unit says: I changed something we cannot see.

Running OCaml from a terminal

Interactively, to try one expression:

```
$ ocaml # or utop, which is friendlier
# 1 + 2 ;;
- : int = 3
```

As a script, for a single file:

```
$ ocaml hello.ml
```

Compiled, which is what we normally want:

```
$ ocamlfind ocamlopt -package str hello.ml -o hello
$ ./hello
```

ocamlopt produces native code; ocamlc produces bytecode. For more than two files, this becomes tedious quickly — which is the point of the next slide.

dune: the build system we will actually use

A Makefile would work. dune is what the OCaml world settled on, and it needs almost no configuration — two small files describe the whole lab:

```
(* dune-project *)      (lang dune 3.0)

(* lib/dune *)          (library (name pqueue_lib) (modules pqueue))
```

Three commands are all we need today:

```
$ dune build                # compile everything
$ dune runtest              # compile and run the tests
$ dune exec bin/solver.exe -- ../mazes/medium.txt --bfs
```

dune works out the dependencies between files itself; there is no list of source files to maintain.

.ml and .mli: the interface is the contract

Each module may carry an interface. The .mli says what exists; the .ml says how.

```
(* pqueue.mli -- the contract *)
type 'a t                (* abstract -- no definition here *)

val empty    : 'a t
val enqueue  : 'a -> 'a t -> 'a t
val dequeue  : 'a t -> ('a * 'a t) option
```

Because 'a t is given *without* a definition, no client can see that a queue is two lists, and therefore no client can build one by hand or depend on the representation. We are free to change it tomorrow.

The compiler checks the .ml against the .mli. This is the whole of today's first half, said in one file — and session 11 returns to it properly.

What we have not said yet

Today was the reading knowledge needed for the lab. The rest arrives when there is a reason for it.

Idea	Today	Properly in
Sum types, exhaustiveness	option only	session 3
Functions as values, fun	mentioned	session 4
Recursion and tail calls	two examples	session 4
map, filter, fold, closures	used once, unexplained	session 5
Lazy evaluation, Seq	the cliffhanger	session 5
Mutation, and when it is safe	ref	session 7
Polymorphism, 'a, inference	'a as <T>	session 10
Modules, signatures, functors	one .mli	session 11

Enough OCaml to build a queue is a surprisingly small language.

The persistence material of this lecture follows
Xavier Leroy's course at the Collège de France.

The amortization and queue material follows
Chris Okasaki, *Purely Functional Data Structures*,
Cambridge University Press, 1998.

Lab Session and Homework

Lab session, and what to do afterwards

In the lab (2 hours).

- *On paper*: the queue's invariant, and why the $O(n)$ reversal is affordable.
- *In Python*: we implement it, then drive the maze solver — breadth-first, the one-line exchange to depth-first, the history view.
- *In OCaml*: the same queue again, against a given `.mli`.
- *And then we break it*: the amortized bound fails, deliberately — session 5 supplies the repair.

Homework.

- Finish the lab, and write the queue a third time in **Java**.
- Small **OCaml exercises** — lists and pattern matching only.
- The **proofs** on the paper sheet: the invariant, the FIFO argument, and the assumption persistence breaks.